

OOPS Principles - 4 [Common in any language].

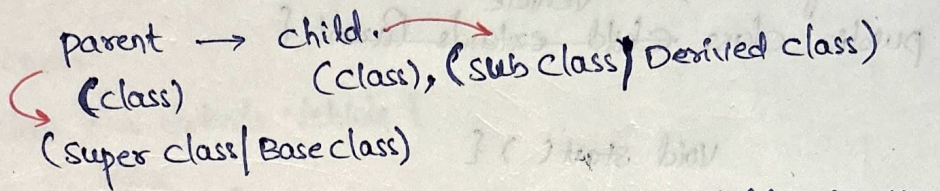
1. Inheritance
 2. Polymorphism
 3. Encapsulation
 4. Abstraction.
- Single Inheritance
Multi Level Inheritance.
Hierarchical Inheritance
Multiple Inheritance
Hybrid Inheritance

}

Don't support in Java with class, support with Interfaces.

1. Inheritance: Relation b/w parent class & child class.

→ Inheriting some features from parent to child.



→ We can inherit some features from parent to child we use keyword extends.

Ex: package com;

```
public class Parent {  
    int a = 10;
```

```
    void m1() {  
        System.out.println("Hi from Parent....");  
    }
```

```
    Psum {
```

```
        Parent p = new Parent();
```

```
        System.out.println(p.a);
```

```
        p.m1();
```

```
        Child ch = new Child();
```

```
        System.out.println(ch.b);
```

```
        ch.m2();
```

```
        ch.m1();
```

```
        System.out.println(ch.a);
```

```
    }
```

Output:

10

Hi from Parent....

20

Hi from child class....

Hi from parent....

10

⇒ Single Inheritance.

Package com;

public class ~~child~~ ^{extends} Parent {

int b = 20;

void m2 () {

sysout("Hi from child class ---");

}

----- x -----

package com;

Vehicle

public class ~~child~~ ^{extends} Parent {

void start () {

sysout("Starting ---");

}

package com;

public class Car extends Vehicle {

void drive () {

sysout("My car is running ---");

}

Package com;

public class Tests {

psvm {

Car car = new Car ();

car.start ();

car.drive ();

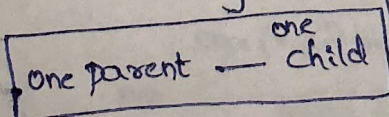
}

Output: Starting ---

My car is running ---

Single Inheritance:

→ One class having one parent & one child.



→ One Java file will have only one class.

Multilevel Inheritance:

Grand father → Father → We → children.

A → B → C → D ----

ex: package com;

```
public class Vehicle {
```

```
    void startmy() {
```

```
        syso("Starting ----");
```

```
    }
```

```
package com;
```

```
public class car extends Vehicle {
```

```
    void drivemy() {
```

```
        syso("My Car is Causing ----");
```

```
    }
```

```
package com;
```

```
public class FuelCar extends car {
```

```
    //(have features of Vehicle & car class)
```

```
    void fuelmy() {
```

```
        syso("My FuelCar is running ----")
```

```
    }
```

```
package com;
```

```
public class Test {
```

```
    psvm {
```

```
        FuelCar fc = new FuelCar();
```

```
        fc.fuel();
```

```
        fc.drive();
```

```
        fc.start();
```

Output: My FuelCar is running ----

my car is Causing ----

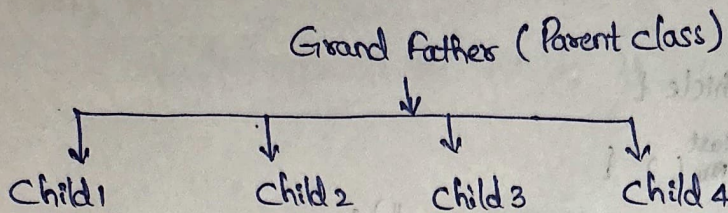
Starting ----

class: 18

09/05/2025

(2020)

Hierarchical Inheritance:



- Child classes have relation with only parent class.
- One parent extended to a multiple children.
- There is no relation b/w child classes.

Ex: package com;

```
public class Car {
```

```
    void drive() {
```

```
        syso("My Car is causing ---");
```

```
    }
```

```
}
```

```
package com;
```

```
public class FuelCar extends Car {
```

```
    void fuel() {
```

```
        syso("Filling Petrol ---");
```

```
    }
```

```
}
```

```
package com;
```

```
public class EVCar extends Car {
```

```
    void charge() {
```

```
        syso("Charging Battery ---");
```

```
    }
```

```
}
```

```
package com;
```

```
public class Test {
```

```
    psum {
```

```
        Car c = new Car() {
```

```
            c.drive();
```



```
FuelCar fc = new FuelCar();
```

```
fc.drive();
```

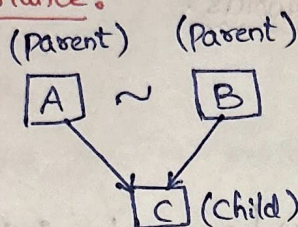
```
fc.fuel();
```

```
EVCar ec = new EVCar();
```

```
ec.drive();
```

```
ec.charge();
```

Multiple Inheritance:



⇒ [C extends A, B]

supports with Interfaces.

- * Multiple Inheritance will ^{not} support in Java with classes,
→ Ambiguity issue (Confused state) - JVM entered to that state.
→ When there is two parent classes, there is a confusion state to access features to child class from both parents.

- * Python supports multiple inheritance, due to it follows order of mentioning parent classes.

- * Diamond Problem - Multiple Inheritance.

→ To overcome this problem, having another concept called

Interfaces.

- * Java will not support to access features from two classes at a time.

Ex: package com;

```
public class Car {
```

```
void driver() {
```

```
    syso("My Car is Cruising...");
```

```
}
```

```
}
```

```
package com;
```

```
public class EVCar {
```

```
void charge() {
```

```
    syso("Charging Battery...");
```

```
}
```

```
}
```


package com;

public class HybridCar extends FuelCar, EVCar {

(error)

}

* Hybrid Inheritance:

→ Cannot support in Java, only supports with interfaces.

This Vs Super Keywords.

* this:

→ It is a keyword in Java.

→ It refers current instance(object) / same class instance.

* Super:

→ It is keyword in Java.

→ It refers to parent class.

Ex:

package com;

```
public class Car {  
    String colour = "Black";  
    void drive() {
```

```
        syso("My Car is running...");  
    }  
}
```

package com;

```
public class FuelCar extends Car {
```

```
    int fuelCapacity = 40;
```

```
    String colour = "Red";
```

```
    void fuel() {
```

```
        syso("Filling Petrol...");
```

```
        syso(this.fuelCapacity); → 40
```

```
        syso(super.colour); → Black
```

```
    } void print() {  
        syso(colour); → Red
```

```
    }  
}
```

Outputs

Red

Black

```
syso(super.colour) Car c = new Car();
```

```
FuelCar fc = new FuelCar();
```

```
fc.print();
```

[Instance variables can access only with objects in main method, but with

"this" keyword, we can access, in same class]

Instance variable without any object }

Ex: package com;

public class Car {

↖ Constructor
Car() { syso ("Car object created ----");
void drive() {
syso ("My Car is running ----");
}
}

package com;

public class FuelCar extends Car {

↗ It can access parameters also.
public FuelCar() { super();
syso ("FuelCar object created ----");
}
void fuel() {
syso ("Filling Petrol ----");
}

[JVM will automatically created even if we not given]
[* ~~this()~~ → This gets error
~~super();~~ → It should be always inside constructor & must be in 1st line]

package com;

package el

public class Test {

psvm {

FuelCar fc = new FuelCar();

Output:

Car object created.
FuelCar object created.

[When we create an object to child class,
automatically parent class object created.]
by JVM.

Car c = new Car();

Output:
Car object created....

[It calls only parent class]

* We cannot call ~~this()~~; in child class.

* Constructor Chaining: calling one constructor inside another constructor.

Ex: package com;
public class Car {
 Car() {
 syso ("Car Object Created ---");
 }
 void drive() {
 syso ("My Car is running...");
 }
}

package com;
public class FuelCar extends Car {
 int fuelCapacity = 40;
 String colour = "Black";
 public FuelCar() {
 this(10); // Constructor which accepts int value.
 syso ("Fuel car object created ---");
 }
 FuelCar (int fuel) {
 this("Red");
 this.fuelCapacity = fuel;
 }
 FuelCar (String colour) {
 this(20, "Brown");
 this.colour = colour;
 }
 FuelCar (int fuel, String colour) {
 this.fuelCapacity = fuel;
 this.colour = colour;
 }
}

Constructor Calling

package com;
public class Test {
 public static void main (String args[]) {
 FuelCar fc = new FuelCar();
 syso (fc.colour);
 syso (fc.fuelCapacity);
 }
}

Output:
Car object created
Fuel Car object created
Red
20
Car object created.

* We can't use same datatypes in different constructors,
we can write in same constructor.

FuelCar (int fuel, int price);


```
void hi() {  
    syso ( "this. fuelCapacity");
```

```
static void bye() {  
    syso (this.fuelCapacity);  
    error we are  
}
```

* [we can't use this(), super(); in static methods
because loaded ~~is~~ with class ~~&~~, at that time
objects were not created] loading time

Ex: